

AI-POWERED LEGAL DOCUMENT GENERATOR

Project Testing : Technical Design Notes

Document Purpose: This technical design note defines the testing strategy, test levels, test cases, AI evaluation methods, security checks, performance validation, user acceptance testing, defect management, and release criteria for the AI-Powered Legal Document Generator. The objective is to verify that the system is functional, secure, reliable, usable, and capable of producing consistent document drafts.

1. Project Testing Overview

Testing is a systematic process used to identify defects, validate requirements, measure system quality, and confirm that the developed application behaves as expected. Because this project combines conventional software components with AI-generated content, testing covers both deterministic software behavior and probabilistic AI behavior.

Testing principle: No generated legal document should be treated as automatically legally correct. The system must support validation and human review, while testing focuses on correctness, consistency, traceability, security, and safe behavior.

2. Testing Objectives

- Verify that all functional requirements are implemented correctly.
- Ensure user inputs are validated and processed correctly.
- Verify that generated documents follow approved templates and required structures.
- Evaluate AI output for relevance, completeness, consistency, and unsupported claims.
- Validate the RAG pipeline and retrieval of approved legal references.
- Verify authentication, authorization, privacy, and secure data handling.
- Measure system performance under expected workloads.
- Identify, track, and resolve defects before release.
- Confirm that the final system meets defined acceptance criteria.

3. Testing Scope

In-Scope Testing

- Frontend forms, navigation, validation, preview, and download functions.
- Backend APIs, business logic, authentication, and database operations.
- AI/LLM prompt processing and generated content structure.
- RAG ingestion, retrieval, ranking, and context integration.
- Legal document template rendering and file generation.
- Security, performance, usability, compatibility, and reliability.
- End-to-end user workflows and acceptance testing.

Out-of-Scope / Controlled Testing

- Determining whether a document is legally sufficient for a specific real-world matter.
- Replacing review by a qualified legal professional.

- Guaranteeing outcomes in courts, tribunals, or government authorities.

4. Testing Strategy

The project follows a layered testing strategy. Testing begins with individual units and progresses through component integration, complete system validation, AI evaluation, and user acceptance.

- Unit Testing
- Integration Testing
- System Testing
- AI/LLM Evaluation
- RAG Testing
- Security Testing
- Performance Testing
- Usability Testing
- Compatibility Testing
- User Acceptance Testing (UAT)

5. Unit Testing

Unit tests validate small, isolated pieces of application logic. They should be automated where practical and executed during development and continuous integration.

- Input validation functions.
- Date, name, address, and required-field validation.
- Document-type selection logic.
- Template field mapping.
- API request and response validation.
- Authentication and authorization helper functions.
- Document formatting utilities.
- RAG chunking and metadata processing functions.
- Error-handling and retry logic.

6. Integration Testing

Integration testing verifies that multiple modules communicate correctly and exchange the expected data.

- Frontend to backend API communication.
- Backend to database integration.
- Backend to AI/LLM service integration.
- RAG retrieval service to AI generation pipeline.
- Template engine to document renderer.
- Authentication service to protected application resources.
- Generated document storage and download workflow.

7. System Testing

System testing validates the complete application against functional and non-functional requirements.

- User login and secure session workflow.
- Document type selection and form submission.
- Complete AI document-generation workflow.
- Document preview and editing.
- Final PDF/DOCX generation where supported.
- Error handling when AI or database services fail.
- Access control between different user roles.
- Audit logging of important system actions.

8. Functional Test Scenarios

Example Test Cases

- **TC-01:** Valid user login should allow access to the application.
- **TC-02:** Invalid credentials should display a safe authentication error.
- **TC-03:** Required document fields should prevent submission when empty.
- **TC-04:** Invalid input formats should be rejected with clear messages.
- **TC-05:** Selecting a document type should load the correct input form.
- **TC-06:** Valid data should trigger the document-generation workflow.
- **TC-07:** Generated output should contain the required sections.
- **TC-08:** Preview should match the generated document structure.
- **TC-09:** Download should produce a readable and correctly formatted file.
- **TC-10:** Unauthorized users should not access protected documents.
- **TC-11:** AI/model failure should result in a controlled error response.
- **TC-12:** Relevant RAG content should be included when available.

9. AI/LLM Testing

AI testing is different from conventional pass/fail testing because the exact wording of a response may vary. Evaluation therefore focuses on measurable quality criteria and safety constraints.

- **Relevance:** Output should address the requested document type and user-provided facts.
- **Completeness:** Required sections should be present.
- **Consistency:** Names, dates, amounts, and other facts should remain consistent throughout the document.
- **Grounding:** Claims based on retrieved knowledge should be supported by the provided context.
- **Hallucination control:** The model should not invent facts, citations, parties, or legal provisions.
- **Instruction following:** The model should follow the approved template and formatting instructions.
- **Safety:** The system should communicate limitations and encourage appropriate legal review.

10. RAG Testing

RAG testing verifies that the system retrieves useful and relevant knowledge from the approved legal knowledge base before generation.

- Verify source ingestion and text extraction.
- Verify document chunking and metadata.
- Verify embedding generation.
- Test semantic search for known legal topics.
- Verify retrieval relevance and ranking.
- Test behavior when no relevant source is found.
- Verify source/version metadata is preserved.
- Check that retrieved context is actually passed to the generation pipeline.
- Test outdated or conflicting knowledge-base entries using controlled datasets.

11. AI Quality Evaluation Metrics

- **Relevance Score:** How closely generated content matches the requested document purpose.
- **Completeness Score:** Percentage of required sections and fields correctly generated.
- **Groundedness:** Degree to which factual/reference-based statements are supported by retrieved context.
- **Consistency:** Agreement of repeated facts across all sections.
- **Format Compliance:** Conformance with the selected document template.
- **Unsupported Content Rate:** Frequency of unsupported or invented factual/legal content.
- **Human Review Score:** Reviewer assessment of overall usefulness and required corrections.

12. Security Testing

- Test login, logout, session expiration, and password handling.
- Verify role-based authorization and access restrictions.
- Test protection against SQL injection and unsafe input.
- Test cross-site scripting (XSS) protections.
- Validate API authentication and authorization.
- Verify sensitive information is not exposed in logs or error messages.
- Check secure storage of API keys and service credentials.
- Verify HTTPS/TLS configuration in deployed environments.
- Test document access to ensure one user cannot retrieve another user's private files.
- Verify audit records for important security events.

13. Performance Testing

Performance testing determines whether the application provides acceptable response times and stability under expected workloads.

- Measure page and API response times.
- Measure document-generation latency.
- Measure RAG retrieval latency.
- Measure AI service response time.
- Test concurrent document-generation requests.

- Observe CPU, memory, database, and storage utilization.
- Test behavior during high load and service timeouts.
- Verify that long-running generation requests provide appropriate progress or status feedback.

14. Usability Testing

- Verify that document creation can be completed through a clear step-by-step workflow.
- Ensure form labels and instructions are understandable.
- Provide clear validation and error messages.
- Make preview and editing functions easy to understand.
- Ensure important warnings and AI limitations are visible.
- Check responsive behavior on common screen sizes.
- Evaluate the system with representative users and record usability feedback.

15. Compatibility Testing

- Test supported desktop browsers such as Chrome, Edge, and Firefox.
- Verify responsive behavior on common mobile/tablet screen sizes.
- Test generated documents in commonly used PDF readers.
- Where DOCX is supported, verify compatibility with standard office software.
- Check consistent fonts, spacing, headings, page breaks, and symbols.

16. Regression Testing

Regression testing is performed after changes to ensure previously working functionality has not been broken.

- Run automated unit tests after code changes.
- Repeat critical API and workflow tests after integration changes.
- Re-test AI output after prompt or model changes.
- Re-test RAG quality after knowledge-base updates.
- Verify document formatting after template changes.
- Maintain a regression test suite for every release.

17. User Acceptance Testing (UAT)

UAT validates whether the system satisfies practical user expectations and project acceptance criteria.

- Users select a document type and complete a realistic request.
- Users verify that all required information appears correctly.
- Users review generated content and identify corrections.
- Users verify preview and download functionality.
- Users provide feedback on clarity, usability, and usefulness.
- Project stakeholders approve the release when acceptance criteria are met.

18. Defect Management

- Record every defect with a unique issue identifier.

- Capture steps to reproduce, expected result, actual result, severity, and evidence.
- Classify defects as Critical, High, Medium, or Low.
- Assign defects to responsible developers.
- Track status from Open → In Progress → Fixed → Retest → Closed.
- Perform root-cause analysis for critical or recurring issues.
- Maintain regression coverage for fixed defects.

19. Test Environment

- **Development:** Used by developers for unit and component testing.
- **Testing/Staging:** Production-like environment for integration, system, security, and performance testing.
- **Production:** Controlled release environment after approval.
- Use separate databases and credentials for each environment.
- Use synthetic or anonymized test data instead of unnecessary real legal/personal information.
- Maintain reproducible configuration and environment documentation.

20. Test Data Management

- Create representative test cases for different document types.
- Include valid, invalid, incomplete, boundary, and unusual inputs.
- Include multilingual or special-character inputs where relevant.
- Test long names, addresses, dates, numbers, and large text fields.
- Use controlled legal knowledge-base samples with known expected retrieval results.
- Avoid using real confidential legal records unless explicitly authorized and properly protected.

21. Release Readiness Criteria

- All critical and high-severity defects are resolved or formally accepted.
- Core functional test cases pass.
- Integration and end-to-end tests pass.
- Security testing has no unresolved critical findings.
- AI evaluation meets the project's predefined quality thresholds.
- RAG retrieval performs acceptably on the evaluation dataset.
- Generated documents meet approved formatting requirements.
- Performance is acceptable for expected workload.
- UAT is completed and approved.
- Technical documentation and deployment configuration are complete.

22. Testing Deliverables

- Test strategy and test plan.
- Functional test cases.
- Unit and integration test results.

- AI evaluation dataset and evaluation report.
- RAG retrieval evaluation report.
- Security and performance test reports.
- Defect/issue tracking report.
- UAT feedback and approval record.
- Final test summary and release recommendation.

23. Continuous Testing and Future Improvements

- Integrate automated tests into CI/CD pipelines.
- Maintain benchmark datasets for AI and RAG evaluation.
- Monitor production quality metrics after release.
- Re-evaluate prompts and models when model versions change.
- Version and re-test legal knowledge sources after updates.
- Introduce automated document-format and schema validation.
- Expand adversarial and security testing as the system evolves.

24. Conclusion

The Project Testing Phase ensures that the AI-Powered Legal Document Generator is reliable, secure, usable, and technically ready for deployment. Testing must cover both traditional software quality and AI-specific risks such as hallucination, weak retrieval, inconsistent output, and unsupported claims. A combination of automated testing, controlled AI evaluation, security validation, performance testing, and human acceptance review provides a strong quality foundation for the project.